Manual Técnico – Marketplace de Tutorías

1. Introducción

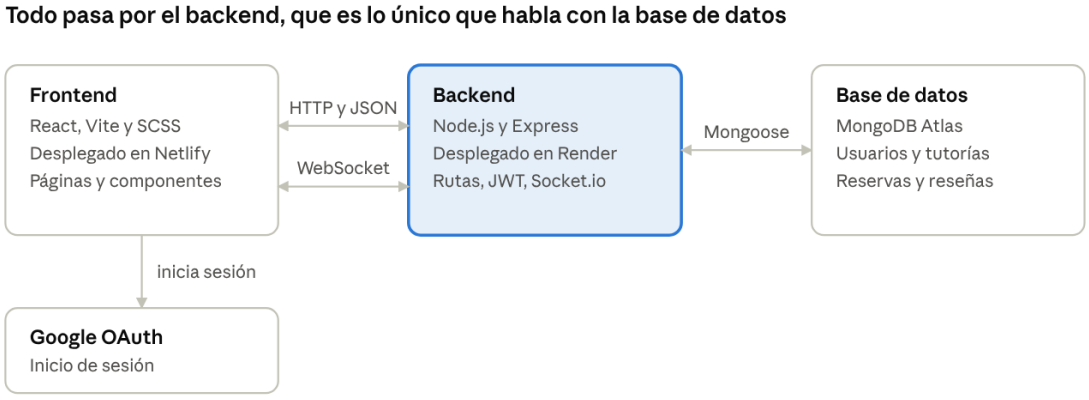
Este Marketplace de Tutorías es una aplicación web donde los estudiantes encuentran tutores, reservan clases y se comunican con ellos, y donde los tutores publican sus ofertas y administran su horario. El proyecto se desarrolló para el curso de Desarrollo Web (Ingeniería en Sistemas, UMG Sede Naranjo) usando el stack MERN: MongoDB, Express, React y Node.js, con TypeScript básico.

Roles del sistema

Rol	Qué puede hacer
Estudiante	Explorar tutorías con filtros, reservar clases viendo el costo antes de confirmar, cancelar reservas, chatear con el tutor y dejar reseñas
Tutor	Publicar ofertas, definir su disponibilidad semanal, aceptar o rechazar reservas, chatear con el estudiante y ver sus reseñas. Necesita ser aprobado por un administrador
Administrador	Aprobar tutores y administrar categorías y subcategorías

2. Arquitectura del sistema

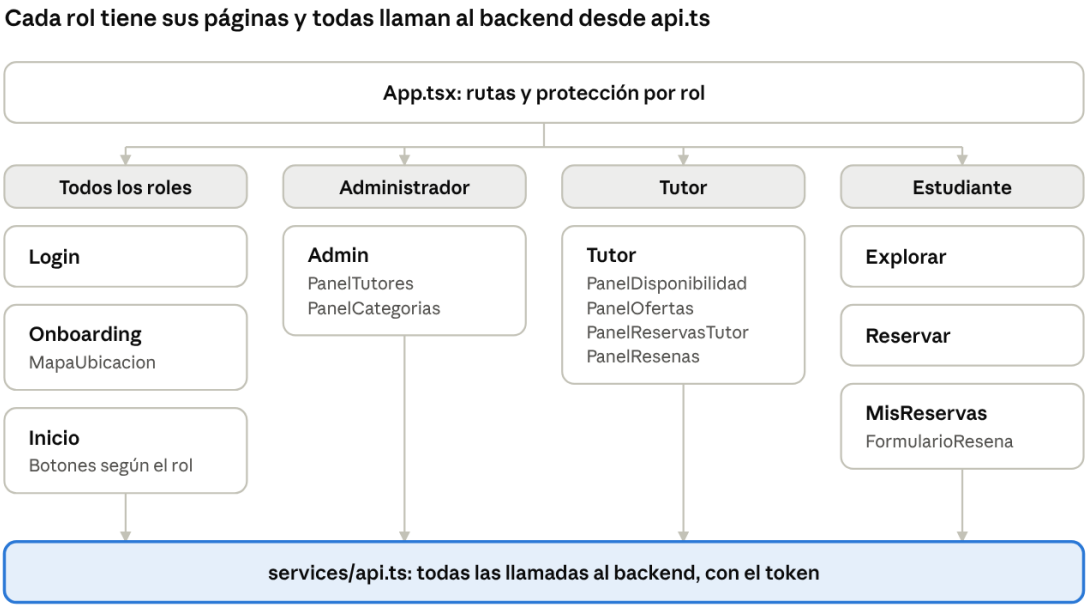
El sistema usa la arquitectura MERN dividida en tres capas: un frontend en React, un backend en Node con Express, y una base de datos MongoDB. El frontend nunca habla con la base de datos; siempre pasa por el backend.



El navegador usa HTTP con JSON para la API y un WebSocket para el chat. El backend usa Mongoose para leer y guardar en Atlas. El inicio de sesión lo resuelve Google desde el navegador.

3. Esquema conceptual de componentes

El frontend se organiza en páginas (una por pantalla) y componentes (piezas que se reutilizan dentro de las páginas). `App.tsx` define las rutas y bloquea el acceso según el rol; cada página arma su pantalla con componentes y todas llaman al backend desde un solo archivo, `services/api.ts`.



4. Descripción de componentes

Cada componente es una función de React con nombre en mayúsculas (la única excepción a la regla `fnc`) y adentro usa funciones con nombre `fnc` para sus eventos.

Páginas

Página	Quién la usa	Qué hace
Login	Sin sesión	Muestra el botón de Google y guarda el token y el usuario al iniciar sesión
Onboarding	Usuario nuevo	Formulario de registro inicial con mapa para marcar la ubicación y las categorías reales de la base de datos
Inicio	Todos	Pantalla principal con los botones que corresponden al rol
Admin	Administrador	Panel con las pestañas para aprobar tutores y administrar categorías
Tutor	Tutor aprobado	Panel con disponibilidad, ofertas, bandeja de reservas y reseñas

Página	Quién la usa	Qué hace
Explorar	Estudiante	Lista de tutorías con filtros por categoría y modalidad
Reservar	Estudiante	Formulario de reserva que muestra el costo calculado antes de confirmar
MisReservas	Estudiante y tutor	Lista de reservas, cancelación y formulario de reseña cuando corresponde

Componentes

Componente	Qué hace	Datos que recibe
MapaUbicacion	Mapa de Leaflet para elegir o mostrar una ubicación	Coordenadas
PanelTutores	Lista de tutores pendientes y aprobados, con botón para aprobar o revocar	Ninguno, consulta la API
PanelCategorias	Crear, editar y eliminar categorías y subcategorías	Ninguno, consulta la API
PanelDisponibilidad	Cuadrícula semanal con bloques de horario que se guardan juntos	Ninguno, consulta la API
PanelOfertas	Formulario y lista de las tutorías del tutor	Ninguno, consulta la API
PanelReservasTutor	Bandeja de reservas para aceptar o rechazar	Ninguno, consulta la API
PanelResenas	Reseñas que ha recibido el tutor	Ninguno, consulta la API
ListaResenas	Promedio y lista de reseñas	`datos` (promedio, total y reseñas)
FormularioResena	Estrellas y comentario para calificar una reserva	`reservaId` y `alGuardar`
Estrellas	Muestra o permite elegir de 1 a 5 estrellas	`valor` y `alCambiar` (opcional)

5. Tecnologías y dependencias

El proyecto usa solo herramientas conocidas del curso. Se evitó a propósito agregar librerías que no se vieron en el curso.

Backend

Paquete	Para qué se usa
Node.js	Entorno donde corre el servidor
Express	Servidor web y rutas de la API
Mongoose	Conexión con MongoDB y definición de los modelos
cors	Permite que el frontend (otro dominio) llame a la API
dotenv	Lee las variables del archivo `.env`
jsonwebtoken	Crea y verifica el token de sesión (JWT)
google-auth-library	Verifica el token que entrega Google al iniciar sesión
socket.io	Chat en tiempo real mediante WebSockets
TypeScript 5 y ts-node-dev	Escribir el código con tipos básicos y reiniciar el servidor al guardar

Frontend

Paquete	Para qué se usa
React	Construcción de la interfaz con componentes y hooks básicos
Vite	Herramienta de desarrollo y de compilación del proyecto
TypeScript	Tipos básicos para datos y componentes
react-router-dom	Navegación entre páginas sin recargar
@react-oauth/google	Botón de inicio de sesión con Google
socket.io-client	Conexión del navegador con el chat del servidor
leaflet	Mapas para elegir y mostrar ubicaciones
sass	Estilos en archivos SCSS

6. Estructura del proyecto

El proyecto está dividido en dos carpetas independientes: backend y frontend. Cada una tiene su propio package.json y se despliega por separado.

1. Entorno Backend (Node.js / Express)

- **.env**: Archivo de configuración para variables de entorno privadas (no se incluye en GitHub por seguridad).
- **src/server.ts**: Archivo principal de arranque del servidor, configuración de middlewares (CORS, Express JSON) y montaje de rutas de la API.
- **src/config/db.ts**: Lógica de conexión a la base de datos de MongoDB.
- **src/models/**: Esquemas de la base de datos (Usuario, Categoría, Tutoría, Disponibilidad, Reserva, Reseña).
- **src/middlewares/auth.ts**: Validaciones de seguridad, decodificación de tokens JWT y protección de rutas por roles.
- **src/routes/**: Controladores que exponen los endpoints de la API (autenticación, categorías, panel de administración, disponibilidad, tutorías, reservas y reseñas).
- **src/utilidades/tiempo.ts**: Funciones auxiliares para el cálculo de fechas, horas, distancias y sesiones.

2. Entorno Frontend (React / Vite)

- **.env**: Variables de entorno del cliente (VITE_API_URL para la conexión al backend y las credenciales de Google Auth).
- **src/main.tsx y App.tsx**: Punto de entrada de la aplicación y configuración del enrutador de páginas.
- **src/pages/**: Vistas principales del sistema (Login, Onboarding, Inicio, Admin, Panel de Tutor, Explorar, Reservar, Mis Reservas y Chat).
- **src/components/**: Componentes visuales modulares y reutilizables (tarjetas de información, paneles, mapa, sistema de estrellas, formularios).
- **src/services/api.ts**: Archivo centralizado que gestiona todas las peticiones y llamadas HTTP al backend.
- **src/utilidades/**: Funciones para dar formato visual a los datos (fechas, dinero y textos de la interfaz).
- **src/styles/**: Archivos de estilos y diseño estructurados en SCSS.

7. Base de datos

La base de datos es MongoDB (en Atlas) y se maneja con Mongoose. Cada colección tiene un modelo en backend/src/modelos. Todas guardan automáticamente la fecha de creación y de edición. Los campos _id los crea MongoDB.

Diagrama entidad-relación

Las seis colecciones se relacionan mediante referencias y no hay datos repetidos entre ellas, salvo el precio que la reserva guarda a propósito.

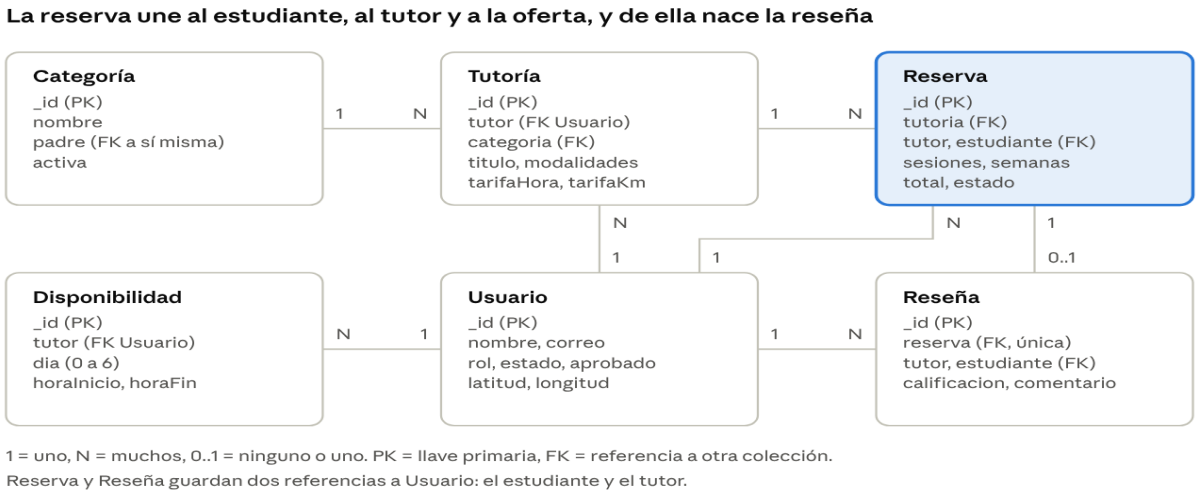


diagrama entidad-relación · 6 colecciones

La lista de categorías dentro de Usuario guarda nombres de categorías de interés, no referencias, por eso no aparece como relación en el diagrama.

8. Endpoints de la API

Todas las rutas empiezan con /api y responden en formato JSON. Si algo falla, devuelven un campo mensaje con la explicación. Excepto el login, todas piden el token en el encabezado Authorization: Bearer token

La columna Acceso indica quién puede usar la ruta: Login (sin sesión), Sesión (cualquier usuario con token), Estudiante, Tutor (aprobado) o Admin.

Autenticación: /api/auth

Método	Ruta	Acceso	Qué hace
POST	`/google`	Login	Verifica el token de Google, crea el usuario si no existe y devuelve el JWT propio
GET	`/perfil`	Sesión	Devuelve los datos del usuario que inició sesión
PUT	`/onboarding`	Sesión	Guarda el registro inicial y el rol elegido (estudiante o tutor)

Administración: /api/admin

Método	Ruta	Acceso	Qué hace
GET	`/tutores?aprobado=true/false`	Admin	Lista tutores aprobados o pendientes
PUT	`/tutores/:id/aprobacion`	Admin	Aprueba o revoca a un tutor

Tutorías: api/tutorias

Método	Ruta	Acceso	Qué hace
GET	`/`	Sesión	Explora ofertas activas de tutores aprobados; filtra por `categoria` (incluye subcategorías) y `modalidad`
GET	`/mias`	Tutor	Lista mis ofertas
GET	`/:id`	Sesión	Detalle de una oferta
POST	`/`	Tutor	Crea una oferta
PUT	`/:id`	Tutor	Edita una oferta; solo su dueño

Reservas: api/reservas

Método	Ruta	Acceso	Qué hace
POST	`/`	Estudiante	Crea una reserva. Valida disponibilidad y traslapes de tutor y estudiante, y calcula el costo
POST	`/?simular=true`	Estudiante	Devuelve el cálculo del costo sin guardar nada
GET	`/mias`	Sesión	Reservas del usuario, como estudiante o como tutor
PUT	`/:id/responder`	Tutor	Acepta o rechaza una reserva pendiente (`aceptar: true/false`)
PUT	`/:id/cancelar`	Participante	Cancela la reserva

Reseñas: api/resenas

Método	Ruta	Acceso	Qué hace
POST	`/`	Estudiante	Califica una reserva aceptada cuya primera clase ya empezó. Una reseña por reserva
GET	`/mias`	Sesión	Ids de las reservas que ya califiqué
GET	`/recibidas`	Tutor	Mis reseñas con promedio y total
GET	`/tutor/:id`	Sesión	Reseñas públicas de un tutor, con promedio y total

9. Autenticación, roles y seguridad

El sistema no guarda contraseñas: el inicio de sesión lo hace Google y el backend solo comprueba que ese inicio sea real.

Flujo de inicio de sesión

1. El usuario presiona el botón de Google en el frontend y Google devuelve un token de identidad.
2. El frontend envía ese token a POST api/auth/Google.
3. El backend lo verifica con google-auth-library(verifyIdToken).
4. Si el correo no existe en la base de datos, se crea el usuario con rol estudiante.
5. El backend crea su propio JWT con el id y el rol del usuario. Su duración se define en la variable JWT_DURACION.
6. El frontend guarda el token y los datos del usuario en localstorage y lo envía en cada petición.

10. Chat en tiempo real

El chat entre estudiante y tutor usa WebSockets mediante Socket.io. A diferencia de una petición normal, la conexión queda abierta, así que el servidor puede enviar un mensaje al otro usuario en cuanto llega, sin que este tenga que recargar la página.

Cómo funciona

1. El servidor Express y Socket.io comparten el mismo servidor HTTP y el mismo puerto.
2. Al abrir el chat, el navegador se conecta con socket.io-cliente a la dirección que indica VITE_API_URL.
3. Cuando un usuario envía un mensaje, el servidor lo recibe y lo reenvía a la otra persona de la conversación.
4. El componente de React recibe el mensaje y lo agrega a la pantalla con useState.

11. Uso de TypeScript

TypeScript es JavaScript con tipos: antes de ejecutar el programa, el compilador revisa que cada dato se use como corresponde y avisa de los errores. Se usa en el backend y en el frontend, pero de forma básica: solo interfaces y los tipos string, number y boolean, sin tipos avanzados.

Dónde se usa

Lugar	Qué se tipa	Ejemplo
Modelos del backend	La forma de cada documento de MongoDB, con una interface	`IUsuario`
Middlewares del backend	La solicitud con los datos del token	`SolicitudAutenticada`
Parámetros y retornos	Qué recibe y qué devuelve cada función	`fncRedondear(valor: number): number`
Interfaces del frontend	Los datos que responde la API	`ReservaDetalle`, `Resena` en `api.ts`
Props de componentes	Los datos que recibe un componente	`PropsEstrellas`
Eventos de React	El tipo de evento de cada formulario o botón	`ChangeEvent`, `FormEvent`, `MouseEvent`

12. Uso de SASS/SCSS y metodología BEM

Los estilos se escriben en SCSS, que es CSS con variables y anidación, y se dividen en un archivo por pantalla o módulo dentro de frontend/scr/styels. Vite compila el SCSS automáticamente gracias al paquete sass.

Archivo	Qué estiliza
`global.scss`	Estilos base de toda la aplicación
`login.scss`, `inicio.scss`, `onboarding.scss`	Pantallas de entrada y registro
`admin.scss`	Panel del administrador
`tutor.scss`	Panel del tutor
`estudiante.scss`	Explorar, reservar y mis reservas
`resenas.scss`	Estrellas, formulario y lista de reseñas

Metodología BEM

Las clases siguen el formato `bloque__elemento--modificador`, así cada nombre dice a qué parte pertenece y no se mezclan estilos entre pantallas.

- Bloque: el componente completo, por ejemplo `resena-form`.
- Elemento: una parte del bloque, por ejemplo `resena-form__boton`.
- Modificador: una variante, por ejemplo `estrellas__estrella--llena`.

Qué aporta SCSS

- **Variables:** el color principal se guarda una vez en `$color-principal` y se usa en todo el archivo, así cambiarlo es modificar una sola línea.
- **Anidación:** los elementos y modificadores BEM se escriben dentro de su bloque con &, sin repetir el nombre completo.
- **Archivos por módulo:** cada pantalla tiene su archivo y se importa en `main.tsx`, por eso los estilos son fáciles de ubicar.
- **Compilación:** el navegador no entiende SCSS; Vite lo convierte a CSS normal al desarrollar y al hacer `npm run build`.

13. Despliegue, variables de entorno y CORS

El frontend está en Netlify y el backend en Render; los dos se actualizan solos cada vez que se hace `git push` al repositorio de GitHub. La base de datos está en MongoDB Atlas.

Parte	Plataforma	Qué se configura
Frontend (React con Vite)	Netlify	Comando `npm run build`, carpeta `dist` y un archivo `_redirects` para que las rutas de React funcionen al recargar
Backend (Node y Express)	Render	Comando de inicio y las variables de entorno
Base de datos	MongoDB Atlas	Usuario de base de datos y acceso de red permitido
Login con Google	Google Cloud	Orígenes autorizados: `localhost` y el dominio de Netlify

Variables de entorno

Variable	Dónde	Para qué sirve
`PORT`	Backend	Puerto del servidor; Render lo asigna solo
`MONGO_URI`	Backend	Cadena de conexión a Atlas
`JWT_SECRETO`	Backend	Clave para firmar los tokens
`JWT_DURACION`	Backend	Cuánto dura una sesión, por ejemplo `2h`
`VITE_API_URL`	Frontend	Dirección del backend: `http://localhost:4000` en desarrollo y la URL de Render en producción
Client id de Google	Frontend y backend	Identifica la aplicación ante Google

Vite solo expone al navegador las variables que empiezan con `VITE_`, y las lee al compilar. Por eso, si se cambia `VITE_API_URL`, hay que volver a desplegar el frontend.

CORS dinámico

El backend no acepta cualquier origen. Compara el dominio de cada petición con la lista de dominios permitidos que se configura en las variables de entorno, y solo responde si está en la lista. Así el mismo código sirve en local y en producción, sin editarlo, y el mismo control se aplica a Socket.io.

14. Instalación y ejecución local

Backend

1. Entrar a la carpeta `backend` e instalar con `npm install`.
2. Crear el archivo `.env` con `PORT`, `MONGO_URI`, `JWT_SECRETO` y `JWT_DURACION`, siguiendo el ejemplo de `.env.example`.
3. Iniciar con `npm run dev`. En la consola debe aparecer "Base de datos conectada" y "Servidor en puerto 4000".

Frontend

1. Entrar a la carpeta `frontend` e instalar con `npm install`.
2. Crear el archivo `.env` con `VITE_API_URL=http://localhost:4000` y el client id de Google.
3. Iniciar con `npm run dev` y abrir la dirección que muestra Vite.